Journal of King Saud University – Computer and Information Sciences 34 (2022) 4269–4283

Contents lists available at ScienceDirect

Journal of King Saud University – Computer and Information Sciences

journal homepage: www.sciencedirect.com

Domain knowledge based genetic algorithms for mobile robot path planning having single and multiple targets

Ritam Sarkar [a], Debaditya Barman [b], Nirmalya Chowdhury [a,*]

[a] Department of Computer Science & Engineering, Jadavpur University, Kolkata 700032, India
[b] Department of Computer & System Sciences, Visva-Bharati, Santiniketan 731235, India

ARTICLE INFO

Article history:
Received 25 May 2020
Revised 12 September 2020
Accepted 10 October 2020
Available online 17 October 2020

Keywords:
Genetic algorithms
Robot path planning
Domain knowledge based operator
Single target path planning
Multiple targets path planning

ABSTRACT

In path planning problem of a mobile robot, the objective is to find an optimal collision free path from a source to single or multiple targets. Domain knowledge based genetic algorithms have been proposed to address the path planning problem having single as well as multiple independent targets. Four new domain knowledge based operators namely "circuit removal operator", "insertion-deletion operator", "refinement Operator", and "target alignment operator" have been introduced in this work. Among these four operators, first three have been used for the path planning problem having single target, whereas all these four operators have been used for path planning problem having multiple independent targets. Proposed methods have been deployed on several simulated environments of varying sizes. From the experimental results, it has been established that the domain knowledge based operators enhance the capability of conventional genetic algorithm. Our proposed method for Mobile Robot's Path Planning Problem having single target has outperformed some previously proposed evolutionary algorithm based methods.

© 2020 The Authors. Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

1. Introduction

Mobile Robot's Path Planning Problem (MRPPP) has been one of the most important research topics from past few years. In a MRPPP, each possible path has a start node and it may contain single or multiple target nodes. There is also an environment through which the Robot traverses. The aim is to get an optimal or near optimal path considering the distance traversed by the robot being the optimization criteria. Robot's environment consists of a number of obstacles. Depending upon the nature of obstacles, MRPPP is of two types: 1. Robot's path planning in an environment where the obstacles are fixed throughout the whole process which is known as a static environment. 2. Robot's path planning in an environment where the obstacles are not fixed. For instance, the obstacles may change their positions and/ or some new obstacles may appear or disappear over the period of time. This is called dynamic environment.

Robot path planning can also be broadly divided into two types (i.e. off-line path planning and on-line path planning) based on the availability of the prior knowledge about the environment. Off-line is a kind of path planning where the prior knowledge about the environment is known in advance whereas in case of on-line path planning, the prior information regarding the environment is unknown, the robot gets the information through some sensors whenever any changes occurred in the environment.

Robot's path planning problem with single target is solvable by several techniques (Zhang et al., 2018). Some of them are: C-Space (Lozano-Perez, 1983), Potential field (Chuang and Ahuja, 1998), A* algorithm (Warren, 1993), Dynamic programming (Willms and Yang, 2006), Voronoi diagram (Takahashi and Schilling, 1989), Soft computing approaches which consist of Artificial Neural Network (Gaudiano et al., 1996), Evolutionary algorithm (Particle swarm optimization (PSO) Nasrollahy and Javadi, 2009), Fuzzy logic (Meléndez et al., 2013) etc. Liu et al. (2017) used ant colony optimization (ACO) algorithm to solve MRPPP. They used grid based environment. Ayari and Bouamama (2017) used dynamic distributed PSO to find optimal collision free path for multiple robots. A hybrid edge detection algorithm based on Fuzzy logic and bacterial algorithm was proposed by Al-Jarrah et al. (2018). Wang et al.

* Corresponding author.
E-mail addresses: nirmalya63@gmail.com, nirmalya.chowdhury@jadavpuruniversity.in (N. Chowdhury).

Peer review under responsibility of King Saud University.

Production and hosting by Elsevier

https://doi.org/10.1016/j.jksuci.2020.10.010
1319-1578/© 2020 The Authors. Published by Elsevier B.V. on behalf of King Saud University.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

(2018) also used PSO for their MRPPP with multiple objectives. A combination of PSO and differential evolution has been used by Zhang et al. (2018) to solve MRPPP. Saraswathi et al. (2018) proposed an hybrid algorithm using Cuckoo search and Bat algorithm to solve MRPPP where the environment is unknown or partially known. Tuba et al. (2018) used brain storm optimization which is a swarm intelligence algorithm for MRPPP with static obstacles. Lamini et al. (2018) improved crossover operator to generate feasible path. Moreover, it prevents premature convergence, as after crossover operation, infeasible paths may get generated due to variable length chromosomes. Orozco-Rosas et al. (2019) developed a hybrid technique based on membrane computing based GA and artificial potential field method to generate collision free optimum path. This edge detection algorithm has been used in a modified GA to return optimum path for mobile robots. Dai et al. (2019) improved ACO for MRPPP by incorporating characteristics of A* technique and MAX–MIN Ant system. Li et al. (2019) formulated MRPPP as a topology optimization problem based on the identification heat transferring path. Luan et al. (2020) proposed a multi-agent A-heuristic technique for mobile robots. A PSO based algorithm to compute shortest collision free path for mobile robot path planning has been proposed by Alam et al. (2020).

Major problems of these approaches are their high computational complexity and the fact that they may stuck at local optima. It seems that, Genetic Algorithm (GA) based method (Tuncer and Yildirim, 2012; Li et al., 2006; Hu and Yang, 2004; Kang et al., 2014) would be very efficient to solve such problems, since it is a multi-dimensional search technique where it deals with multiple solutions simultaneously (Murthy and Chowdhury, 1996). GA has been applied on a number of other domains also. Few of them are-(Deaven and Ho, 1995; Reeves, 1995; Hou et al., 1994; Murthy and Chowdhury, 1996). It may be noted that Robot's path planning is a NP-Hard problem (Canny, 1988).

The path of mobile robot may contains multiple targets. This can be of three types. a) Each of the targets is dependent on the other targets, b) The targets are independent of each other, c) Combination of type-a and type-b (Dong et al., 2016) i.e. some of the targets are dependent and rest are independent. At first, a Domain Knowledge Based Genetic Algorithm for Single Target has been proposed to deal with Robot's path planning problem. Then it has been extended into a Domain Knowledge Based Genetic Algorithm for Multiple Targets (DKBGAMT) that can handle the problem of multiple targets of type b.

The rest of the paper has been organized as follows. Section 2 and 3 contain the detailed description of the problem and the proposed method, respectively. Experimental results using different simulated environments can be found in Section 4, and the concluding remarks have been stated in Section 5.

2. Problem description

In this paper, we have addressed the problem of single robot path planning having single target where each possible path consists of a start node, a target node, and a number of intermediate nodes in between them. For this purpose, we have developed a number of heuristics and added those into our proposed conventional genetic algorithm. These heuristics use various environment related information and act as local search strategies. The goal of these local search strategies is to enhance the capability of conventional genetic algorithms in terms of finding better solutions in lesser time. These facts have been established experimentally. Further, we have addressed another problem of single robot path planning for multiple independent targets where a possible path comprises of a start node, multiple target modes, and some number of intermediate nodes between the start node and the first target node as well as between any two consecutive target nodes. Moreover, the target nodes can appear in any order as they are independent of each other. We have considered that robot traverses on a simulated environment which is as an orderly numbered grid based environment. The objective is to get an optimal or near optimal path considering the distance traversed by the robot as the optimization criteria. There are certain assumptions those have been considered throughout the entire work as mentioned below.

(i) All the simulated environments are static in nature.
(ii) The velocity of the robot is fixed.
(iii) The complete information regarding the environment has been assumed to be known in advance.
(iv) Since the obstacle boundary consists of the actual boundary of the obstacles and the minimum safety distance needed for the robot, the mobile robot has been considered as a point throughout this study (Hu and Yang, 2004).

3. The proposed methodologies

We have proposed GA based approaches to solve robot's path planning problem where each path has a single source along with single or multiple targets. Section 3.1 presents our proposed method for planning robot's path having single target and Section 3.2 presents the method for planning robot's path having multiple targets.

3.1. The proposed methodology for single target

In this work, we have proposed a GA based optimization technique to find the optimal (i.e. shortest) path for a robot to reach from the start node (S) to target node (T) in static environment without making any collision with the obstacles. GA is a stochastic search procedure inspired by the principle of natural genetic system. In GA every solutions are encoded as chromosomes and some predefined number of chromosomes constitute the population. Genetic operators (i.e. Selection, Crossover, and Mutation) are applied on the individuals of the population on every iterations or generations until some termination criteria are satisfied. At the end of each generation, Elitist strategy is applied to copy the best chromosome of the previous generation into the current generation. Fitness function is used to evaluate how good a particular chromosome with respect to other chromosomes in the population. This fitness function may be a minimized or maximized one depending upon the problem scenario.

3.1.1. Representation of environments

Grid based decomposition has been used in this work to decompose the robot's environment into a collection of orderly numbered grids which are equal in shape and size. We have taken the grids as rectangular (Tuncer and Yildirim, 2012; Li et al., 2006; Hu and Yang, 2004; Kang et al., 2014) in shape. As shown in Fig. 1(a) the grids are of basically two types: colored grid and blank grid. Colored grids represent obstacles and the grids through which robot can move freely are the blank grids. The start node (S) and target node (T) are known in advance.

3.1.2. Representation of path

Each of the paths in this Robot's path planning problem has been considered as a chromosome. There exist different types of representations for the path of mobile robot. Such as decimal coded (Tuncer and Yildirim, 2012; Li et al., 2006; Hu and Yang, 2004), binary coded (Nagib and Gharieb, 2004) etc. In this paper, decimal coded path representation has been used where possible paths are of various lengths. In each path, the first node is the start

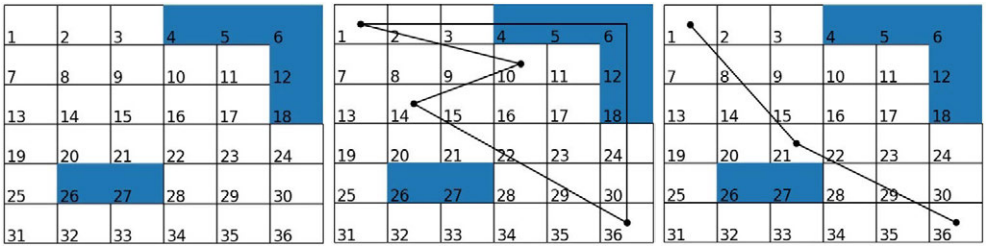


Fig. 1. (a) Robot's environment, (b) Robot's path, (c) Example for mutation.

node and the last node is the target node. If every segments of a path passed through the blank grids, then only the path can be considered as a feasible path otherwise it is an infeasible path. As shown in Fig. 1(b) the path (1,10,14,36) is a feasible path whereas the path (1,6,36) is an infeasible path.

3.1.3. Initial population

The individuals or paths in the initial population, also known as chromosomes have been generated randomly in such a way that none of the individuals are infeasible. Presence of infeasible individuals may reduce the convergence rate. For a specific problem, as the population size increases, the searching process becomes more diverse but it introduces extra computational cost. So, it is very important to choose the population size in such a manner that keeps a balance between the diversity in the searching process and the computational cost. Moreover, standard guideline for choosing the appropriate value of population size for a given environment does not exist in the literature. Also there does not exist any criteria which can measure the complexity level of an environment by just examining the size of the environment, position of the source and destination, shape, size, and the location of the obstacles present in the environment. In addition to these, there exist no such method that yields appropriate size of population for an environment of a particular complexity level. So, the only solution is to use different population sizes for an environment and select the one that provides best result within an acceptable limit of computational time. Likewise, we have used different population sizes for each of the environments and reported the population size for which we have obtained the result which is acceptable in terms of computational time.

3.1.4. Fitness

Fitness function is used to evaluate the goodness of a chromosome in a population. Let P be a path containing n nodes from starting node to target node. Since distance has been considered as the optimization criteria, the objective function (f) (as shown in Eq. (1)) for path P needs to be minimized.

$$f = \sum_{i=1}^{n} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2} + penalty$$
$$\text{Where, penalty} = \begin{cases} \beta, \text{if path } P \text{ is infeasible} \\ 0, \text{if path } P \text{ is feasible} \end{cases} \quad (1)$$

In Eq. (1), x_i and x_{i+1} are the x coordinates of the $i-th$ and $(i+1)-th$ node of path P, respectively. Similarly, y_i and y_{i+1} are the y coordinates of the $i-th$ and $(i+1)-th$ node of path P and *penalty* represents the penalty score for the path P. It is equal to the maximal path segment (i.e. β) possible in an environment if the path P is found to be an infeasible path, otherwise zero.

Since the purpose of this path planning problem is to obtain the shortest feasible path from start node to target node, we need to minimize the objective function f presented in Eq. (1). It may be noted that, since penalty score increases the fitness value, the probability of selecting an infeasible path becomes reduced.

3.1.5. Genetic operators

There are three genetic operators. These are selection, crossover, and mutation. These three genetic operators have been described below.

3.1.5.1. Selection. Selection operator is based on the concept of "survival of the fittest". In other words, the probability of the selection of a particular chromosome is proportional to the fitness value of that chromosome. Chromosomes are selected from a population to create a mating pool whose size has been taken as same as the size of the population.

3.1.5.2. Crossover. In crossover, two parent chromosomes exchange information between themselves and generate two off-springs for the next generation. Here in this work, we have used single point crossover where the single point has been chosen randomly from the parent chromosome having shorter length except the start node and target node. Otherwise, after performing crossover there may be a chance of having multiple start nodes or target nodes in one of the off-springs whereas no start node or target node in the other off-spring.

Parent 1	0	12	15	20	35	67	90	100	150
Parent 2	0	10	13	17	75	150			
Offspring 1	0	12	15	20	75	150			
Offspring 2	0	10	13	17	35	67	90	100	150

Parent 1	0	12	15	20	35	67	90	100	150
Parent 2	0	10	13	17	75	150			
Offspring 1	0	12	15	20	35	67			
Offspring 2	0	10	13	17	75	150	90	100	150

As shown in the example-1 of crossover, parent 1 and parent 2 are the parent chromosomes where 0 is the starting node and 150 is the target node. Suppose, the fourth position has been selected randomly for crossover. As a result, the rest of the parts (i.e. the parts after the fourth position) have been exchanged between the parent chromosomes in order to generate the off-springs. But according to the crossover shown in the example-2 of crossover,

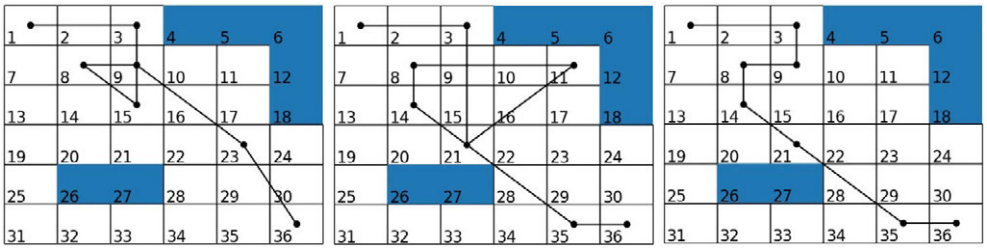


Fig. 2. (a) Example for visible circuit, (b) Path before Circuit Removal, (c) Path after Circuit Removal.

if the sixth position is selected for crossover, then the offspring 1 does not contain target node (i.e. 150) whereas the offspring 2 contains the target node two times. For that reason, the crossover point needs to be chosen from the parent chromosome with shorter length and it can not be the start node or target node.

3.1.5.3. Mutation. Mutation operator introduces some genetic diversities into the population in order to explore the solution space without being stuck at local optima. The mutation operator used in this study is based on the mutation operator proposed in Tuncer and Yildirim (2012). Here in this method, the mutation operator has been modified a little bit as shown in Algorithm 1. For example, consider a path P (1,21,36) as in Fig. 1(c) and if we apply the mutation operator on the path, then it may select any node except the start node and target node. Suppose 21 has been selected for mutation. The neighbours of 21 are 20, 14, 15, 16, 22, 28, 27, and 26. Now 21 will be replaced by one of the nodes from the set {21, 20, 14, 15, 16, 22, 28} which produces the best fitness value. Since the nodes 26 and 27 are obstacles, these are not included in the mentioned set.

Algorithm 1. Mutation

input : P is a path or chromosome,
P_{mut} is the predefined mutation probability,
n is the length of path P.
output: Path P

1 **for** $i \leftarrow 2$ **to** $n-1$ **do** `// i.e. except start node and target node of path` P
2 $P_{rand} \leftarrow$ random number in the interval $[0, 1]$;
3 **if** $P_{rand} \leq P_{mut}$ **then**
4 Replace the node $P[i]$ with its neighbor grids (except the colored grid and the nodes already present in path P) and find out the fitness value for each path (including the previous path i.e. the path having node $P[i]$);
5 Replace $P[i]$ with the grid causing minimum fitness value;
6 **end**
7 **end**

According to the mutation in Tuncer and Yildirim (2012), for each chromosome the mutation operator selects only one node randomly and then generates a random number (say *rand*). The mutation operation is performed if $rand \leqslant P_{mut}$ where P_{mut} is the predefined mutation probability. So, in this type of mutation, for each chromosome at most one node can participate in the mutation operation if the aforementioned condition is satisfied. As a result, more number of iterations or generations are required to explore the solution space which may decrease the convergence rate. To avoid this problem, here in this work every nodes in a path (except the start node and target node) get chance to participate in the mutation operation if the above stated condition is satisfied, thereby reducing the number of iterations.

3.1.6. Domain knowledge based operator

Sometimes, due to the complex alignments and shapes of obstacles in an environment, it becomes very difficult to get an optimal or near optimal path by the conventional GA. In these circumstances, the introduction of some local search strategies reinforce the conventional GA by not letting it converges into the local optima. These local search strategies help the search process to converge into some local or global optima, on the other hand, GA is responsible for finding the most probable regions where the global optima may exist. As a result, the effort of conventional GA along with the local search strategies results in better optimization (Kala, 2012; Kala et al., 2009). These local search strategies are also known as domain knowledge based operators (DKBOs) as they are based on some heuristics which are only applicable to the domain on which they are applied. Three Domain Knowledge Based Operators (DKBOs) have been introduced in this work. These are Circuit Removal operator, Insertion-Deletion operator, and Refinement operator. These operators help to replace a path having higher length with a shorter one by performing some minimal modifications on that path based on some heuristics as stated below.

3.1.6.1. Circuit removal. Circuit Removal operator detects and removes circuits from a path. Usually, if there are multiple occurrences of nodes in a path or chromosome, then circuit occurs. But here in this scenario, sometimes though there is no repetitions of nodes in a chromosome, it may contain some circuits. Circuits are of three types: visible circuit, hidden circuit, and combination of visible and hidden circuit. These have been described below.

Visible Circuit:-

If circuit occurs only due to the repetitions of nodes in the encoded chromosome, then it can be considered as a visible circuit. "*Visible*" in the sense that the circuit can be detected by just observing the encoded chromosome. For instance, consider the path $(1, 3, 9, 15, 8, 9, 23, 36)$ as in Fig. 2(a). Here circuit occurs only due to the multiple occurrences of node 9. So, after applying circuit removal it becomes $(1, 3, 9, 23, 36)$.

Hidden Circuit:-

Sometimes, a path or encoded chromosome may contain some circuits even though there are no repetitions of nodes. These circuits have been considered as hidden circuits. '*Hidden*' in the sense that the circuit can't be detected only by just observing the encoded chromosome. This type of circuit can be of two types.

Type 1:Circuit due to the intersection between two non-adjacent path segments.

Type 2:A path segment passes through a node that is also part of that path.

For example: consider the path (1,3,21,11,8,14,35,36) in Fig. 2 (b). Though there are no repetitions of any nodes, it contains two circuits of each of the aforementioned types. It contains circuit of type 1, due to the intersection between two non adjacent path segments (3,21) and (11,8). Since the path segment (14,35) passes through the node 21 which is a part of the path, it also contains circuit of type 2. After the application of our proposed Circuit Removal operator, the path becomes (1,3,9,8,14,21,35,36) as shown in Fig. 2(c).

Combination of visible and hidden circuit:-

This type of circuit occurs due to the multiple occurrences of nodes and hidden circuits. The algorithm for circuit removal operator has been stated in Algorithm 2.

Algorithm 2. Circuit Removal Operator

input : P is the path containing n nodes
output: Path P

Insert all the intermediate nodes into the path P, that will not change the structure of path P;
$n_1 \leftarrow$ Length of the updated path P;
$count_node \leftarrow 1$; `// count_node is an temporary variable`
while $count_node \leq (n_1 - 1)$ **do**
  if $P[count_node]$ *occurs more than once in* P **then**
    $start_index \leftarrow$ index of first occurrence of $P[count_node]$ in P;
    $end_index \leftarrow$ index of the last occurrence of $P[count_node]$ in P;
    delete $[P_{start_index+1}, \cdots, P_{end_index}]$ from P;
    count_node $\leftarrow$ count_node + 1;
  end
  else
    count_node $\leftarrow$ count_node + 1;
  end
end

3.1.6.2. Insertion-deletion operator. Tsai et al. (2011) applied the concept of *Add* and *Deletion* operator to modify robot path. In *Add* operator, a path has been modified based on a smoothing technique called B-spline which adds a node between two nodes. According to the *Deletion* operator proposed by them, a randomly selected node is deleted. Our proposed Insertion-Deletion operator consists of two sub operators: Insertion and Deletion operator. Here the deletion operator selects two adjacent path segments and checks whether the starting node of the first segment and the last node of the second segment can be joined without having any obstacles in between them. For instance, let (P_i, P_j) and (P_j, P_k) are two adjacent path segments of path P then this operator will delete the node P_j, if P_i and P_k can be connected without colliding with any obstacles (i.e. colored grids). Utility of Deletion operator is to increase convergence rate. But sometimes, application of Deletion operator results in a path having very few number of nodes that can cause premature convergence. For this purpose, Insertion operator has been introduced here. Insertion operator selects a path segment and then inserts a node in between that selected path segment. After insertion, the inserted node is fitted into proper position after the applications of other operators on it. Algorithm 3, describes the algorithm for Insertion-Deletion operator.

Algorithm 3. Insertion-Deletion operator

input : P is a path or chromosome,
P_{in} is the predefined insertion probability,
P_d is the predefined deletion probability.
output: P

Select *Insertion* or *Deletion* with 50% probability;
if *Insertion is selected* **then**
  for *For all path segment in* P **do**
    $r \leftarrow$ a random number in the range [0, 1];
    if $r \leq P_{in}$ **then**
      Randomly selects a node in between that selected path segment;
      Insert the node in between the selected path segments in the path P;
    end
  end
end
else
  for *all two adjacent path segments in* P **do**
    `// Let` (P_i, P_j) `and` (P_j, P_k) `are two adjacent path segments.`
    $r \leftarrow$ a random number in the range [0, 1];
    if $r \leq P_d$ **then**
      if (P_i, P_k) *is a path segment which does not intersect with any colored grid* **then**
        Delete P_j from the Path P.
      end
    end
  end
end

For example, consider the environment shown in Fig. 3(a) and (b). The path shown in Fig. 3(a) is a near optimal path which has been generated by applying only deletion operator along with the other operators, but the application of insertion operator introduces an additional node in the path and makes the path optimal as shown in Fig. 3(b).

3.1.6.3. Refinement operator. The refinement operator used in this work is based on the refinement operator used in Li et al. (2006) and the principle behind this operator is that the length of hypotenuse in a right angle triangle must be less than the summation of the lengths of other two sides. Algorithm 4 describes the Refinement Operator. We have also experimentally shown that the proposed refinement operator works more efficiently than the operator used in Li et al. (2006) in terms of finding a shorter path.

Algorithm 4. Refinement Operator

input : P is a path or chromosome.
output: Path P

1 **for** *all two adjacent path segments in* P **do** // Let (P_i, P_j) and (P_j, P_k) are two adjacent path segments.
2 **if** *these two path segments create a right angle* **then**
3 **if** (P_i, P_K) *do not intersect with any colored grid* **then**
4 Delete node P_j from the path;
5 **end**
6 **end**
7 **else**
8 Find all the intermediate nodes between the path segments (P_i, P_j) and insert them in the proper order starting from P_{i+1} to P_j i.e. $(P_{i+1}, ..., P_j)$;
9 Find all the intermediate nodes between (P_j, P_k) and insert them in the proper order starting from P_{k-1} to P_j i.e. $(P_{k-1}, ..., P_j)$;
10 $X_i \leftarrow P_{i+1}$;
11 $X_j \leftarrow P_{k-1}$;
12 **while** $(X_i != P_j)$ *and* $(X_j != P_j)$ **do**
13 **if** (X_i, X_j) *do not intersect with any colored grid* **then**
14 Delete P_j from path P;
15 Insert X_i after P_i in path P;
16 Insert X_j before P_k in path P;
17 **end**
18 **else**
19 $X_i \leftarrow$ the next element of $[P_{i+1}, ..., P_j]$;
20 $X_j \leftarrow$ the next element of $[P_{k-1}, ..., P_j]$;
21 **end**
22 **end**
23 **end**
24 **end**

For example, consider path (1,20,24,36) as in Fig. 4(a). In this path two adjacent path segments (20,24) and (24,36) create a right angle. Now, since path segment (20,36) collides with obstacles, we have inserted all the intermediate nodes in the above mentioned two path segments. As a result, these two path segments turn into (21,22,23,24) and (30,24) according to line 8 and 9 of Algorithm 4. Now, since (21,30) does not collide with any obstacles, at first 24 has been deleted from the path and then 21 has been inserted after 20 and the node 30 before 36, as stated in line 10 to line 24 of Algorithm 4. As a result, the path becomes (1,20,21,30,36) as shown in Fig. 4(b). But if we apply the refinement operator proposed in Li et al. (2006) on the path shown in Fig. 4(a), we will get the path shown in Fig. 4(c), where the generated path is (1,20,23,30,36), because in Li et al. (2006), the checking regarding the existence of any feasible path segments (i.e. the path segment that has not been collided with any obstacles) between the path segments (20,24) and (24,36) have been started from the intermediate nodes which are adjacent to 24 (i.e. the node where the path segments (20,24) and (24,36) have generated a right angle). As a consequence, it gets the path segment (23,30) which has not been collided with any obstacles and finally generates the path (1,20,23,30,36) which has larger length than the path obtained by the refinement operator used in this work.

3.1.7. Elitist strategy

The purpose of the elitist strategy is to keep the best chromosome of the previous generation into the current generation. Because, the best chromosome (i.e. the elite) might be lost if they are not selected or if crossover or mutation alters them. The method has been described in the Algorithm 5.

Algorithm 5. Elitist Strategy

input : The population of chromosomes at $i-th$ generation i.e. g_i after performing Selection, Crossover, Circuit Removal, Mutation, Insertion-Deletion Operator, and Refinement Operator at i-th iteration, e is the elite chromosome upto generation g_{i-1}.
output: The resulted population of chromosome at $i-th$ generation i.e. g_i, The elite chromosome e.

1 $W_i \leftarrow$ the worst chromosome in the generation g_i;
2 **if** W_i *is worse than* e *in terms of their fitness value* **then**
3 Replace W_i in g_i by e;
4 **end**
5 $e \leftarrow$ the best chromosome in the generation g_i;

3.1.8. Termination condition

Though there is no such universal stopping criteria for GA (Murthy and Chowdhury, 1996), the algorithm will be terminated when either one of the following three conditions will be satisfied:

(i) The total no of generation exceeds 100.
(ii) The algorithm converges to the optimal solution within 100 iterations or generations.
(iii) The fitness value of the best chromosome remain unchanged for around 50 consecutive generations.

3.1.9. Flowchart

The flowchart of the proposed method has been depicted in Fig. 5.

3.2. The proposed methodology for multiple targets

The problem of planning the path of a mobile robot containing multiple targets has three following types. a) Multiple targets, where each target is dependent on the other targets, b) Multiple targets, where targets are independent of each other, and c) Combination of type-a and type-b (i.e. some targets are dependent and rest of them are independent). If all the targets or some of them are

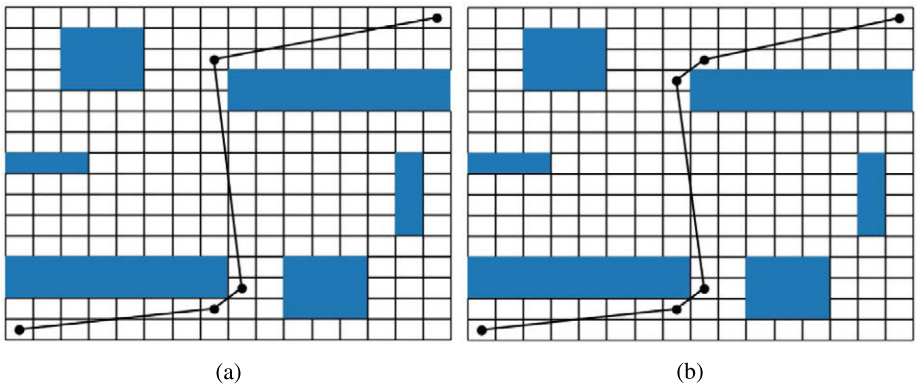


Fig. 3. (a) Path before Insertion, (b) Path after Insertion.

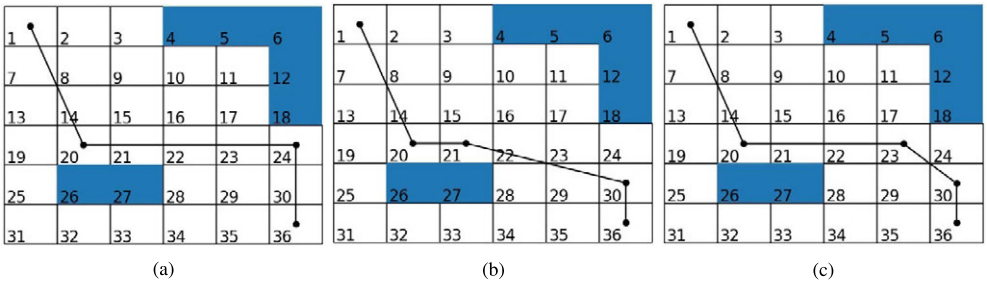


Fig. 4. (a) Path before Refinement Operator, (b) Path after Refinement Operator, (c) Example for Refinement operator in Li et al. (2006).

dependent, then proper ordering of the targets need to be maintained in the planned path. As a result, the objective is to retrieve the shortest path from the source to the first target, then from the first target to second target and so on (one after another) in the required order. For instance, suppose there is a robot at location say X and its job is to carry an object say O_1 from X to another location Y. After that, the robot has to pick up another object O_2 from location Y and carry it to another location Z. In this problem, the source location is X and the targets are Y and Z, respectively. So, we do not have to take any trouble for setting up the order of the targets, we just need to plan a path from X to Y and then from Y to Z. But if there is no such requirement of maintaining any particular order, then the objective is to obtain a path which should start from X and contains two targets Y and Z in such a fashion that minimizes the overall length of the path. So, for type (a) and (c), we can plan the path sequentially (i.e. from X to Y and then Y to Z) but for the type (b), we need to consider all the targets simultaneously, since we do not know the order of the targets which will produce the path having shortest length. Our proposed DKBGAMT can handle the problem of type b. For type a and type c, our proposed GA based method for single target can be applied sequentially (i.e. from start node to 1st target node, then 1st target node to 2nd target node and so on) according to some specified order of target nodes. But, here in this work, we have addressed the problem of type b and to the best of our knowledge, our proposed method is the first attempt to solve the said problem.

3.2.1. Representation of environment

Here, the environment representation is similar to the representation considered for the path with single target described in Section 3.1.1 The only difference is that, this environment contains a source (S) and multiple Targets $\{T_1, T_2, T_3, \ldots, T_n\}$ instead of single target.

3.2.2. Representation of path

Each path contains a source and a set of n Targets $\{T_1, T_2, T_3, \ldots, T_n\}$ which can appear in any order. As mentioned in the Section 3.1.2, if any of the path segments collides with obstacles then the path will be considered as the infeasible path, otherwise it will be a feasible path.

3.2.3. Initial population

As mentioned in the Section 3.1.3 each of the paths of the initial population has been generated randomly in such a manner that all the generated paths meet the feasibility criteria.

3.2.4. Fitness

The technique to evaluate fitness of a path having multiple targets has been taken as the same as mentioned in Section 3.1.4.

3.2.5. Genetic operators

As mentioned before, there exist three types of genetic operators. These are Selection, Crossover, and Mutation. Out of these three operators, the Selection and Mutation operators have been implemented in the same manner as they have been done for the problem of single target (referred to Section 3.1.5).

3.2.5.1. Crossover. Here in this work, every possible path contains the start node and all the predefined Target nodes, but not necessarily in any specific order. For this purpose, before performing crossover operation, the path has been divided into all possible path segments. For instance, if there is a path say P_1 $(S, \ldots, T_{11}, \ldots, T_{21}, \ldots, T_{i1}, \ldots, T_{n1})$ contains n Targets where T_{i1} is the $i-th$ Target of path P_1 and this $i-th$ Target of P_1 may not be same as that of $i-th$ Target of another path say P_2 (i.e.T_{i2}). Here dotted line indicates intermediate nodes. So, the path segments for the path P_1 will be $(S, \ldots, T_{11}), (T_{11} \ldots, T_{21}), \ldots,$

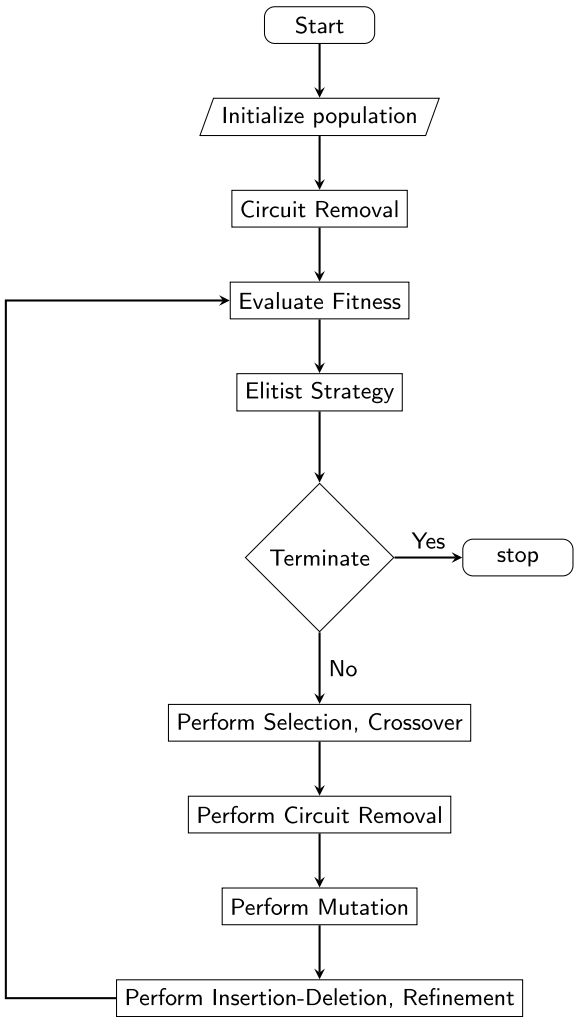


Fig. 5. Flowchart of the proposed algorithm.

$(T_{(i-1)1},\ldots,T_{i1}),(T_{i1},\ldots,T_{(i+1)1}),\ldots$, and $(T_{(n-1)1},\ldots,T_{n1})$. Therefore, if there are n number of Targets, then the number of path segments will be equal to n. Now, one-point crossover as in Section 3.1.5.2 has been performed among the path segments of path P_1 and P_2, if they fall under one of the three cases as mentioned in Sarkar et al. (2020), followed by reordering and merging of the newly generated path segments according to their original order in the paths.

3.2.6. Domain knowledge based operators

Similar to the crossover operator, for all the DKBOs (e.g. Circuit Removal, Insertion-Deletion Operator, and Refinement Operator), at first any path say P_1 having n Targets has been divided into n segments and then DKBOs have been applied on each of these path segments. After that, they have been merged together according to their original order. Here in this study, we have introduced a new DKBO named "Target alignment operator" to plan a path with multiple targets.

3.2.6.1. Target alignment operator. Due to the Mutation, Insertion-Deletion, and Refinement operator, we may get a path where one target may appear more than once. We have proposed a new DKBO named Target alignment operator to deal with this problem. This operator converts such a path into a new path where each target appears only once. Before applying this operator, we need to find the location of the first occurrence of each and every target nodes and store them into an array named *initial_target_positions*. This mechanism can be of three following types of cases.

Case 1:

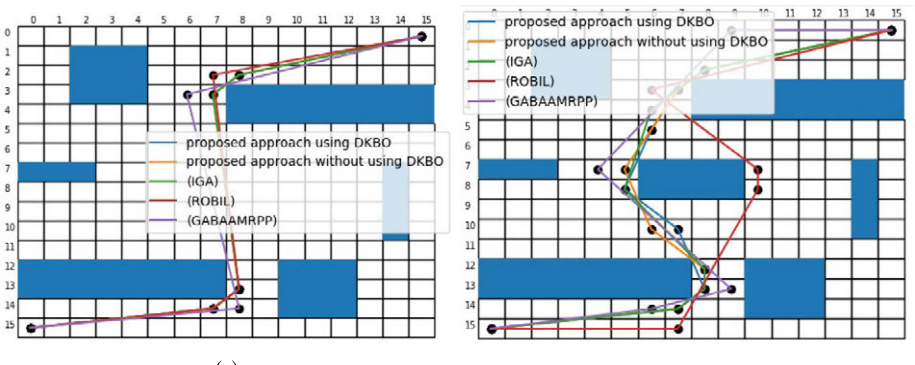
Let (0,5,8,10,14,8,23,18) be a path where 0 is the start node and the target nodes are: 8 and 18, so the *initial_target _positions* = (3,8). Now, since the number of occurrences of target node 8 is two and there is no other target nodes between the location of target node 8 in *initial_target_positions* and the second occurrence of the node 8, the nodes form the 4th position (i.e. the node after the first occurrence of node 8) to the 6th position (i.e. the second occurrence of target node 8) have been deleted. As a result, we have got the path (0,5,8,23,18) where the number of occurrence of each and every target node is exactly one and the length is shorter than the previous one.

Case 2:

Consider the path (0,5,8,23,30,10,18,24,29,30) where 0 is the start node and the target nodes are 8, 30, and 18. So, the *initial_target_position* = (3,5,7) and there exist multiple occurrences of the target node 30 and this is also the last node. In this these circumtances, delete all the nodes from the node after 3rd target (i.e. the second last target node 18) to the last node. As a result, we have got the path (0,5,8,23,30,10,18), where the number of occurrences of each target nodes is exactly one and the length is also shorter than the previous path.

Case 3:

Suppose there is a path (0,5,8,23,18,9,12,15,10,8,24,29,30) where 0 is the start node and the target nodes are 8,18,15, and 30. So, the *initial_target_position* = (3,5,8,13) and the target node 8 has been appeared multiple times. Moreover, there exist multiple



(a) (b)

Fig. 6. (a) Environment 1, (b) Environment 2.

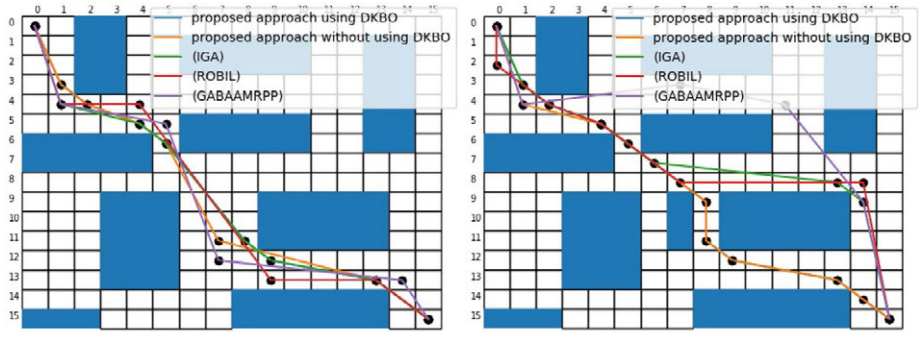


Fig. 7. (a) Environment 3, (b) Environment 4.

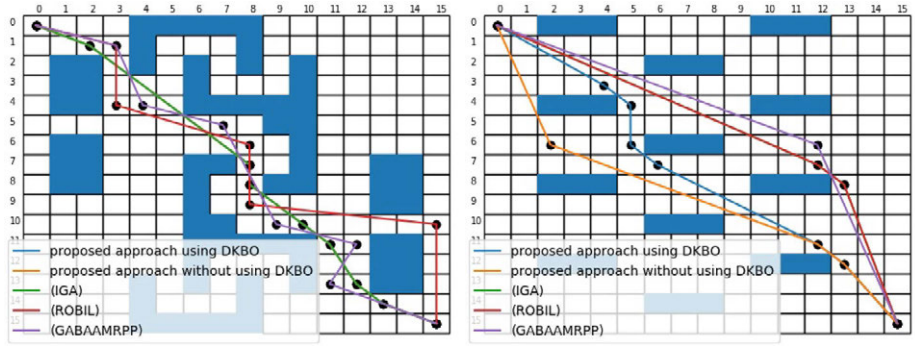


Fig. 8. (a) Environment 5, (b) Environment 6.

target nodes between the first and the second occurrence of 8 and it is not also the last node. Now, consider node 5 (i.e. the node positioned just before the 1st occurrence of node 8) and node 18 (i.e. the target node after the 1st occurrence of node 8) as $segment_1$ (i.e. $segment_1$ = (5,18)) and then the node 15 (i.e. the target node before the 2nd occurrence of node 8) and 24 (i.e. the node just after the 2nd occurrence of node 8) as $segment_2$ (i.e. $segment_2$ = (15,24)). Note that a path segment (say $segment_1$) will be feasible if it does not intersect with any of the obstacles. Depending on the feasibility of the path segment, following cases may exist.

Case 3a:

If both $segment_1$ and $segment_2$ are feasible then compute the fitness value for both the segments. After that, if the $segment_1$ has higher fitness value then delete node 8 and 23 (i.e. delete all the nodes from the 1st occurrence of node 8 up to the node which is just before the next target node), otherwise, delete the node 10 and 8 (i.e. delete all the nodes after the target node 15 till the 2nd occurrence of target node 8). If both $segment_1$ and $segment_2$ have same fitness value then delete either (8,23) or (10,8) randomly.

Case 3b:

If either $segment_1$ or $segment_2$ is feasible then delete node 8 and 23 if $segment_1$ is infeasible or delete node 10 and 8 if $segment_2$ is infeasible.

Case 3c:

Suppose $segment_1$ and $segment_2$ both are infeasible. In that case, there can exist multiple target nodes between the first and the second occurrence of the node 8 and it may not also be the last node. In these circumtances, we have taken separate path for each of the occurrences of target node 8. These are: P_1 = (0,5,23,18,9,12,15,10, 8,24,29,30) and P_2 = (0,5,8,23,18,9,12,15,10,24,29,30). Finally the path having least fitness value has been chosen.

3.2.7. Elitist strategy

Elitist Strategy for this proposed DKBGAMT has also been employed in the same manner as that for single target (referred to Section 3.1.7). The only difference is that, there is a DKBO named Target alignment operator which has been applied at the end of Mutation, Insertion-Deletion, and Refinement operations.

4. Experimental results

Following sections describe the experimental results for the robot's path containing single as well as multiple targets. Section 4.1 narrates the experimental results for single target and the experimental results for multiple targets have been stated in Section 4.2.

4.1. Experimental result for the robot's path containing single target

Based on the size of the environment, this section is further divided into two sections (i.e. Section 4.1.1 and 4.1.2). Section 4.1.1 and 4.1.2 present the Experimental results for the robot's path containing single target on mid-scale environments and large-scale environments, respectively. In addition to that, there is another section (i.e. 4.1.3) where we have presented an overall discussion on single robot path planning.

4.1.1. Experimental result for the robot's path containing single target on mid-scale environments

The proposed method has been deployed on several environments. It may be noted that, some of these environments are also used in some previous work (Tuncer and Yildirim, 2012) on this topic. This has been facilitated the comparison of our experimental

results with that of the GABAAMRPP (Lamini et al., 2018), IGA (Tuncer and Yildirim, 2012), ROBIL (Kang et al., 2014), and the proposed method without using DKBO to solve the same problem. The values of different operators have been used in all the experiments are as follows: Mutation probability-0.1, Crossover probability-1.0, Circuit Removal probability-1.0, Insertion-Deletion probability-0.30 for Deletion and 0.20 for Insertion and Refinement operator probability-1.0. Experimental results can be found in the following Tables 1–6. In these tables, we have reported method's name, population size, best path length, average best path length, average number of generations, and average time to converge in 25 experimental runs. The convergence time also includes the time required to generate the initial population. Environment 1 to 6 are represented in Fig. 6(a),(b); Fig. 7(a),(b); Fig. 8(a).(b) and their experimental results are presented in Tables 1–6, respectively.

From the experimental results presented in Tables 1–6, it can be observed that our proposed method has been succeeded to return shorter path in all the cases and the average best path length is also better than the other approaches. These facts demonstrate the effectiveness of our proposed approach over the other approaches along with conventional Genetic Algorithms (i.e. Proposed method without using DKBO).

4.1.2. Experimental result for the robot's path containing single target on large-scale environments

In this section, we have demonstrated the effectiveness of our proposed method by comparing it's performance with GABAAMRPP (Lamini et al., 2018), IGA (Tuncer and Yildirim, 2012), ROBIL (Kang et al., 2014), and the proposed method without using DKBO on the large-scale environments (i.e. environment – 7 and environment – 8). These environments are shown in Fig. 9 and Fig. 10 and the corresponding experimental results are shown in Table 7 and Table 8, respectively. Based on the experimental results shown in Table 7 and 8, it can be said that, our proposed method has succeeded to find the path having shortest length for both environment – 7 and environment – 8. Although the convergence time for environment – 7 is on the higher side, our proposed method has consistently performed well on large-scale as well as mid-scale environments.

Table 1
For the environment-1.

Method	Population Size	Best path length	Average best path length	Average number of generations	Average time to converge (s)
GABAAMRPP Lamini et al. (2018)	20	28.73	28.73	7	9.64
ROBIL Kang et al. (2014)	30	27.77	32.00	29	7.85
IGA Tuncer and Yildirim (2012)	60	27.22	27.67	15	2.46
Proposed method without using DKBO	20	27.22	27.22	23	1.78
Proposed method with DKBO	10	27.22	27.22	10	0.54

Table 2
For the environment-2.

Method	Population Size	Best path length	Average best path length	Average number of generations	Average time to converge (s)
GABAAMRPP Lamini et al. (2018)	50	31.65	32.01	11	23.90
ROBIL Kang et al. (2014)	70	30.75	32.18	34	21.93
IGA Tuncer and Yildirim (2012)	60	28.71	29.64	25	4.43
Proposed method without using DKBO	50	28.64	29.02	52	6.24
Proposed method with DKBO	30	28.56	28.78	22	3.05

Table 3
For the environment-3.

Method	Population Size	Best path length	Average best path length	Average number of generations	Average time to converge (s)
GABAAMRPP Lamini et al. (2018)	40	24.83	26.14	8	16.98
ROBIL Kang et al. (2014)	50	24.24	25.65	26	10.53
IGA Tuncer and Yildirim (2012)	80	22.86	24.20	20	4.89
Proposed method without using DKBO	40	22.76	23.75	32	4.67
Proposed method using DKBO	20	22.42	22.43	28	3.42

Table 4
For the environment-4.

Method	Population Size	Best path length	Average best path length	Average number of generations	Average time to converge (s)
GABAAMRPP Lamini et al. (2018)	40	26.24	26.42	8	18.48
ROBIL Kang et al. (2014)	60	25.37	26.37	31	18.01
IGA Tuncer and Yildirim (2012)	100	24.20	25.08	17	6.72
Proposed method without using DKBO	60	23.30	24.52	47	8.34
Proposed method using DKBO	30	22.83	24.05	30	3.99

Table 5
For the environment-5.

Method	Population Size	Best path length	Average best path length	Average number of generations	Average time to converge (s)
GABAAMRPP Lamini et al. (2018)	40	24.74	25.14	12	58.53
ROBIL Kang et al. (2014)	100	26.61	40.96	19	23.60
IGA Tuncer and Yildirim (2012)	80	21.85	22.56	19	28.90
Proposed method without using DKBO	40	21.85	22.02	38	17.00
Proposed method using DKBO	10	21.85	21.85	23	6.51

Table 6
For the environment-6.

Method	Population Size	Best path length	Average best path length	Average number of generations	Average time to converge (s)
GABAAMRPP Lamini et al. (2018)	50	22.90	22.90	5	9.25
ROBIL Kang et al. (2014)	50	22.58	23.04	10	1.63
IGA Tuncer and Yildirim (2012)	100	22.58	22.80	16	2.26
Proposed method without using DKBO	60	22.52	22.55	42	4.49
Proposed method using DKBO	50	22.05	22.31	35	3.93

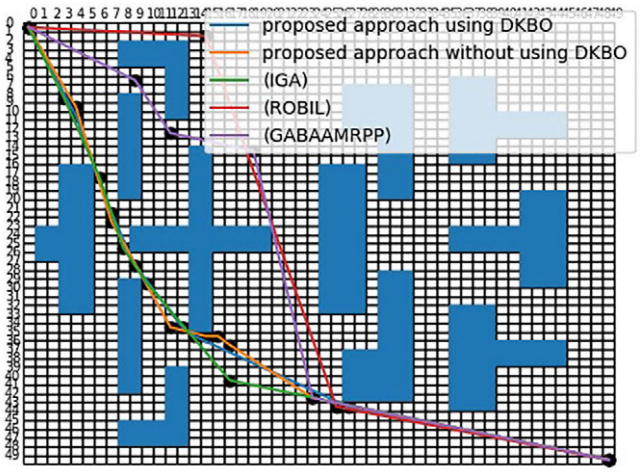


Fig. 9. Environment 7.

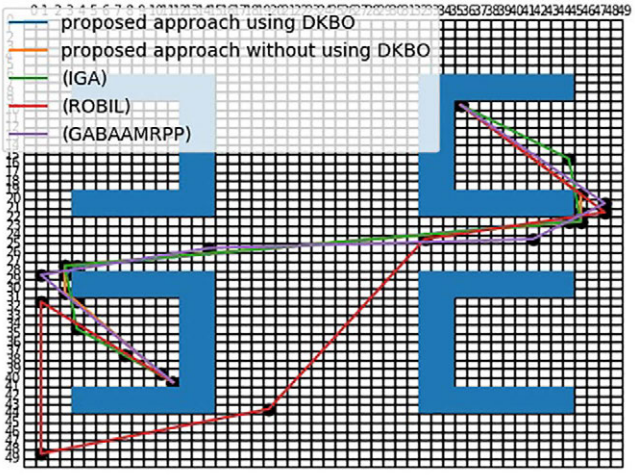


Fig. 10. Environment 8.

4.1.3. Overall discussion for single target path planning

All the experimental results shown in Table 1 to Table 8 suggest that, our proposed method has outperformed the other methods. It has managed to find out the best path for all the environments and the time to converge is also lesser than the other compared methods for most of the environments. Least average best path length indicates that our proposed method has consistently discovered better paths for all the environments. When more than one methods managed to converge into the same solution, the average path length and the convergence time can be used to select the best method. For instance, the best obtained path for environment-1 and environment-5 are same for IGA, proposed method without using DKBO, and the proposed method using DKBO. But, the values for the average best path length and the convergence time for our proposed method are lesser than that for the other approaches which demonstrates the superiority of the proposed method. Some interesting observations have been reported in the following section.

(i) Since our proposed method needs some additional computation due to DKBO, it seems that it's convergence time would be higher than the other approaches. But, the proposed method requires significantly lesser population size, as a consequence, it has managed to converge within lesser time for most of the environments. Generally, as the population size increases, it introduces more variation into the population which helps to converge into a better solution. But, our proposed DKBOs have introduced adequate variation even with lesser population size and that has resulted in finding a better path with considerable less population size and less convergence time. So, by spending the extra computational cost for DKBOs, we are gaining a lot in terms of a better path even within a shorter time.

(ii) As mentioned in Kang et al. (2014), the initial population of Feasible Population based GA (FGA) contains only the feasible paths, whereas the initial population of Classical GA is generated randomly irrespective of their feasibility. As a result, the time required to generate the initial population in FGA is relatively higher than the Classical GA and it varies with the complexity of the environment. For some complex environment FGA may take significant amount of time to generate the initial population. Out of all methods those have been compared in this work, only ROBIL follows the

Table 7
For the environment-7.

Method	Population Size	Best path length	Average best path length	Average number of generations	Average time to converge (s)
GABAAMRPP Lamini et al. (2018)	50	79.21	83.33	31	204.72
ROBIL Kang et al. (2014)	250	82.21	85.44	13	45.22
IGA Tuncer and Yildirim (2012)	150	77.02	88.23	32	80.83
Proposed method without using DKBO	80	76.94	84.16	45	89.34
Proposed method with DKBO	50	76.04	79.17	71	89.24

Table 8
For the environment-8.

Method	Population Size	Best path length	Average best path length	Average number of generations	Average time to converge (s)
GABAAMRPP Lamini et al. (2018)	50	81.11	81.35	17	793.68
ROBIL Kang et al. (2014)	300	84.15	95.13	53	899.84
IGA Tuncer and Yildirim (2012)	60	78.24	80.08	55	276.80
Proposed method without using DKBO	50	76.89	78.47	61	245.01
Proposed method with DKBO	30	75.72	75.98	58	171.20

initial population of classic GA whereas others adopt FGA's mechanism to generate initial population. On the other side, ROBIL also maintains an edge-bank containing feasible path segments whose size increases at every iteration and there is a step at the beginning of every iteration i (where, $i \geqslant 2$) which requires a number of access to this edge-bank. Moreover, the number of feasible paths in the edge-bank varies from one environment to other. As a result, ROBIL has been converged faster than other algorithms in some environments (i.e. environment-6, environment-7).

(iii) According to the mutation operator used in this work, every genes or nodes in the chromosome get a chance to participate in the mutation operation if some predefined criteria is fulfilled as mentioned in Section 3.1.5.3. But it seems that putting too much variation in mutation operator can degrade the effect of the crossover operator applied in the previous step. The proposed method without using DKBO is basically the IGA (Tuncer and Yildirim, 2012) with a difference in mutation operation. In the mutation operation used in IGA, only one node gets the chance to participate provided some predefined criteria is fulfilled and it has been observed from the experimental results that our proposed method without using DKBO performs better than IGA in terms of the population size, path length, and computational time. So, it seems the mutation operator used in this work improves the overall performance instead of degrading it.

(iv) The experimental results shown in Section 3.1.2. indicate that our proposed method performs equally well for large-scale environments as in case of mid-sized environments. This fact establishes the scalability of the proposed method in terms of the size of the environments.

(v) Fig. 11 presents a comparison among ROBIL (Kang et al., 2014), IGA (Tuncer and Yildirim, 2012), the proposed method with and without using DKBOs. All the graphs shown in the Figure 12 are fitness value vs. number of iteration for the best path found in 25 experimental runs. Here each graph contains four curves, one for each of the compared methods along with the proposed method using DKBO. In all the cases the proposed method using DKBO outperforms the others in terms of better fitness value. Since the nature of the fitness function of GABAAMRPP (Lamini et al., 2018) is different from above mentioned four methods, it has not been included in any of the fitness value based comparisons.

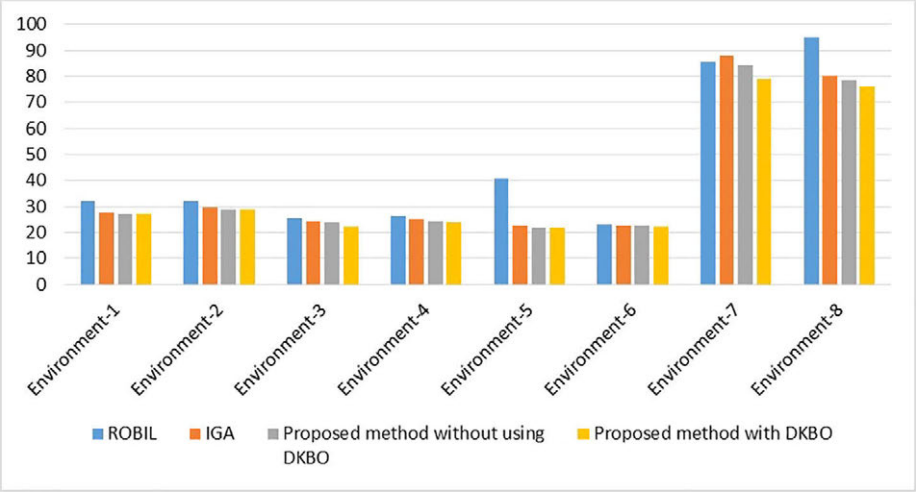


Fig. 11. Average fitness value based comparison.

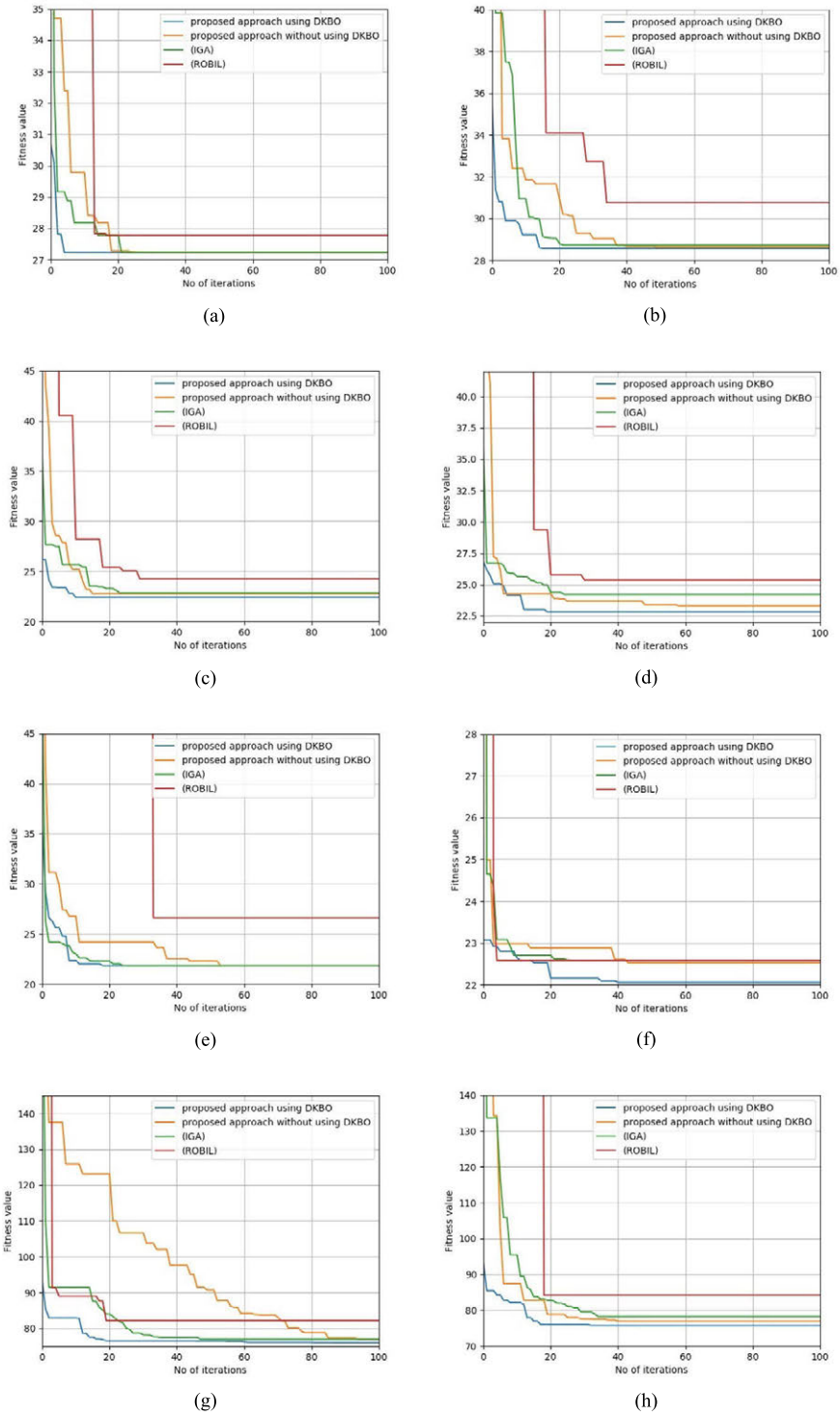


Fig. 12. Comparison between the proposed method using DKBO and without using DKBO for (a) environment 1, (b) environment 2, (c) environment 3, (d) environment 4, (e) environment 5, (f) environment 6, (g) environment 7, and (h) environment 8.

4.2. Experimental result for Robot's path containing multiple targets

We have applied our proposed DKBGAMT for all the mid-sized environments which have been used for single target path problem. The experimental results have been reported in Table 9. Fig. 13(a)–(f) represent the environment 1, environment 2, environment 3, environment 4, environment 5, and environment 6, respectively. The red colored nodes (as shown in Fig. 13) represent the target nodes and the green colored node represents the start node. The experimental results are shown in Table 9 which contains environment number, population size, the number of infeasible solutions, best path length, and the average number of generations in order to get the optimal path in 25 experimental runs. The values of different type of probabilities for all the

Table 9
for the environments shown in Fig. 13.

Environment	population size	The number of infeasible solution	Best path length	The number of generations
Environment 1	70	0	46.86	23
Environment 2	100	0	47.38	31
Environment 3	100	0	29.44	27
Environment 4	50	0	29.91	37
Environment 5	150	0	25.64	35
Environment 6	50	0	25.51	39

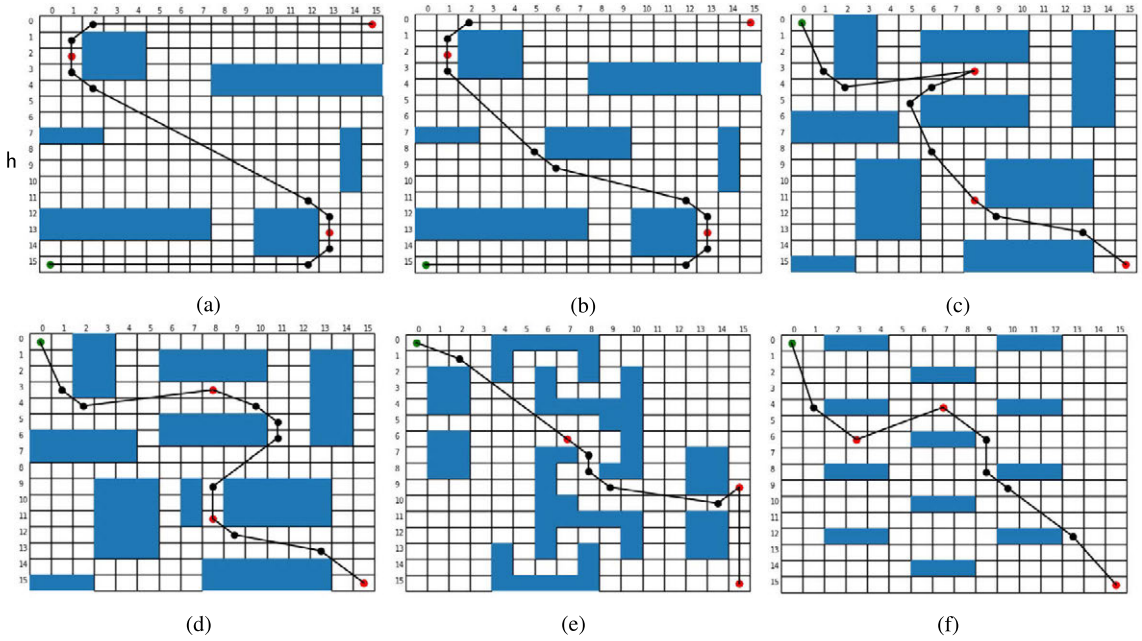


Fig. 13. Proposed DKBGAMT for (a) environment 1, (b) environment 2, (c) environment 3, (d) environment 4, (e) environment 5, and (f) environment 6.

operators are taken as same as mentioned in section Section 4.1. The best paths found by the proposed method are also shown in Fig. 13.

5. Conclusion

Based on the experimental results, we can conclude that our proposed method outperforms other existing methods for finding optimal path for robot's path planning problem for single target. Our proposed method has provided better results in terms of finding shortest (i.e. best) path for all the environments consistently. Moreover, our proposed method has been succeeded to find better path than the regular GA. So, it can be claimed that our proposed DKBO based GA have enhanced the regular GA. We have also extended our proposed method for the robot's path containing a single target into DKBGAMT for handling the problem of multiple targets. These targets are independent of each other which implies that the targets can appear in any order within the path. Therefore, the objective of our proposed DKBGAMT is to order all target nodes and select the appropriate intermediate nodes between the source node and the first target node as well as between any two consecutive target nodes simultaneously. In this manner, we can obtain the optimal or near optimal path between a given source and a set of target nodes. The efficacy of this proposed DKBGAMT has also been established by the experimental results those we have obtained for several simulated environments. However, as mentioned in the problem description, the simulated environments involved in this work are static in nature. Therefore, our proposed method can't tackle dynamic environments. Moreover, there exist a number of other important parameters like safety, smoothness, energy consumption etc. for robot path planning problem apart from the path length. Thereby development of a DKBO based Multi-objective GA for dynamic environment can be considered as the future scope of this research. Biologically inspired multi-objective optimization algorithms can also be explored to solve the path planning problem.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Al-Jarrah, R., Al-Jarrah, M., Roth, H., 2018. A novel edge detection algorithm for mobile robot path planning. J. Robot. 2018..
Alam, M.S., Rafique, M.U., Khan, M.U., 2020. Mobile robot path planning in static environments using particle swarm optimization. arXiv preprint arXiv:2008.10000..
Ayari, A., Bouamama, S., 2017. A new multiple robot path planning algorithm: dynamic distributed particle swarm optimization. Robot. Biomimetics 4, 8.
Canny, J., 1988. The complexity of robot motion planning. MIT Press.
Chuang, J.H., Ahuja, N., 1998. An analytically tractable potential field model of free space and its application in obstacle avoidance. IEEE Trans. Syst., Man, Cybern. Part B (Cybernetics) 28, 729–736.
Dai, X., Long, S., Zhang, Z., Gong, D., 2019. Mobile robot path planning based on ant colony algorithm with a* heuristic method. Front. Neurorobot. 13, 15.
Deaven, D.M., Ho, K.M., 1995. Molecular geometry optimization with a genetic algorithm. Phys. Rev. Lett. 75, 288.

Dong, Y.F., Xia, H.M., Zhou, Y.C., 2016. Disordered and multiple destinations path planning methods for mobile robot in dynamic environment. J. Electr. Comput. Eng. 2016..

Gaudiano, P., Zalama, E., Coronado, J.L., 1996. An unsupervised neural network for low-level control of a wheeled mobile robot: noise resistance, stability, and hardware implementation. IEEE Trans. Syst., Man, Cybern. Part B (Cybernetics) 26, 485–496.

Gia Luan, P., Thinh, N.T., 2020. Real-time hybrid navigation system-based path planning and obstacle avoidance for mobile robots. Appl. Sci. 10, 3355.

Hou, E.S., Ansari, N., Ren, H., 1994. A genetic algorithm for multiprocessor scheduling. IEEE Trans. Parallel Distribut. Syst. 5, 113–120.

Hu, Y., Yang, S.X., 2004. A knowledge based genetic algorithm for path planning of a mobile robot, in: Robotics and Automation, 2004. Proceedings. ICRA'04. 2004 IEEE International Conference on, pp. 4350–4355..

Kala, R., 2012. Multi-robot path planning using co-evolutionary genetic programming. Expert Syst. Appl. 39, 3817–3831.

Kala, R., Shukla, A., Tiwari, R., 2009. Comparative analysis of intelligent hybrid systems for detection of pima indian diabetes, in: Nature & Biologically Inspired Computing, 2009. NaBIC 2009. World Congress on, IEEE. pp. 947–952..

Kang, B.Y., Xu, M., Lee, J., Kim, D.W., 2014. Robil: robot path planning based on pbil algorithm. Int. J. Adv. Rob. Syst. 11, 147.

Lamini, C., Benhlima, S., Elbekri, A., 2018. Genetic algorithm based approach for autonomous mobile robot path planning. Procedia Comput. Sci. 127, 180–189.

Li, B., Liu, H., Su, W., 2019. Topology optimization techniques for mobile robot path planning. Appl. Soft Comput. 78, 528–544.

Li, Q., Zhang, W., Yin, Y., Wang, Z., Liu, G., 2006. An improved genetic algorithm of optimum path planning for mobile robots, in: Intelligent Systems Design and Applications, 2006. ISDA'06. Sixth International Conference on, pp. 637–642..

Liu, J., Yang, J., Liu, H., Tian, X., Gao, M., 2017. An improved ant colony algorithm for robot path planning. Soft. Comput. 21, 5829–5839.

Lozano-Perez, T., 1983. Spatial planning: a configuration space approach. IEEE Trans. Comput., 108–120

Meléndez, A., Castillo, O., Valdez, F., Soria, J., Garcia, M., 2013. Optimal design of the fuzzy navigation system for a mobile robot using evolutionary algorithms. Int. J. Adv. Rob. Syst. 10, 139.

Murthy, C.A., Chowdhury, N., 1996. In search of optimal clusters using genetic algorithms. Pattern Recogn. Lett. 17, 825–832.

Nagib, G., Gharieb, W., 2004. Path planning for a mobile robot using genetic algorithms. IEEE Proc. Robot., 185–189

Nasrollahy, A.Z., Javadi, H.H.S., 2009. Using particle swarm optimization for robot path planning in dynamic environments with moving obstacles and target, in: Computer Modeling and Simulation, 2009. EMS'09. Third UKSim European Symposium on, pp. 60–65..

Orozco-Rosas, U., Montiel, O., Sepúlveda, R., 2019. Mobile robot path planning using membrane evolutionary artificial potential field. Appl. Soft Comput. 77, 236–251.

Reeves, C.R., 1995. A genetic algorithm for flowshop sequencing. Comput. Oper. Res. 22, 5–13.

Saraswathi, M., Murali, G.B., Deepak, B., 2018. Optimal path planning of mobile robot using hybrid cuckoo search-bat algorithm. Procedia Comput. Sci. 133, 510–517.

Sarkar, R., Barman, D., Chowdhury, N., 2020. A cooperative co-evolutionary genetic algorithm for multi-robot path planning having multiple targets. Computat. Intell. Pattern Recogn., 727–740 Springer.

Takahashi, O., Schilling, R.J., 1989. Motion planning in a plane using generalized voronoi diagrams. IEEE Trans. Robot. Autom. 5, 143–150.

Tsai, C.C., Huang, H.C., Chan, C.K., 2011. Parallel elite genetic algorithm and its application to global path planning for autonomous robot navigation. IEEE Trans. Industr. Electron. 58, 4813–4821.

Tuba, E., Strumberger, I., Zivkovic, D., Bacanin, N., Tuba, M., 2018. Mobile robot path planning by improved brain storm optimization algorithm. In: 2018 IEEE Congress on Evolutionary Computation (CEC). IEEE, pp. 1–8.

Tuncer, A., Yildirim, M., 2012. Dynamic path planning of mobile robots with improved genetic algorithm. Comput. Electr. Eng. 38, 1564–1572.

Wang, B., Li, S., Guo, J., Chen, Q., 2018. Car-like mobile robot path planning in rough terrain using multi-objective particle swarm optimization algorithm. Neurocomputing 282, 42–51.

Warren, C.W., 1993. Fast path planning using modified a* method, in: Robotics and Automation, 1993. Proceedings., 1993 IEEE International Conference on, pp. 662–667..

Willms, A.R., Yang, S.X., 2006. An efficient dynamic system for real-time robot-path planning. IEEE Trans. Syst., Man, Cybern. Part B (Cybernetics) 36, 755–766.

Zhang, J.H., Zhang, Y., Zhou, Y., 2018. Path planning of mobile robot based on hybrid multi-objective bare bones particle swarm optimization with differential evolution. IEEE Access 6, 44542–44555.